Token Write Amplification in LLM-Mediated State Stores

Shijun Yang
University of Science and Technology of China
Hefei, China
yangx7@mail.ustc.edu.cn

Shuheng Gao
University of Science and Technology of China
Hefei, China
shhgao@mail.ustc.edu.cn

Chaomei Yan
University of Science and Technology of China
Hefei, China
yanchaomei@mail.ustc.edu.cn

Guoli Wei
University of Science and Technology of China
Hefei, China
weiguoli@mail.ustc.edu.cn

Yongkun Li
University of Science and Technology of China
Hefei, China
ykli@ustc.edu.cn

Abstract

LLM-based applications increasingly rely on persistent state stores to maintain durable memories across long-running interactions. Unlike conventional storage systems, these stores often invoke LLMs on the write path to transform raw context into semantic state, shifting maintenance cost from bytes written to tokens processed. We introduce *token write amplification* (TWA), the ratio of write path LLM tokens to raw context tokens written, to quantify this byte-invisible cost. Across five state store configurations and seven agent workloads, native write paths incur an average TWA of 8.00×, showing that semantic state maintenance can consume far more tokens than the context being stored. By accounting for writes at the level of semantic units, we find that 54.9% of native TWA is associated with semantics already covered by the existing state, indicating that redundant semantic processing is a substantial write path cost. Motivated by this observation, we propose a lightweight, backend-agnostic *semantic coalescing buffer* that groups related writes before storage. Across the same workloads, semantic coalescing reduces average TWA from 8.00× to 3.02× without backend modifications, demonstrating that semantic overlap is a key optimization target for LLM-mediated state stores.

CCS Concepts

• **General and reference** → **Metrics**; • **Information systems** → **Information systems applications**.



This work is licensed under a Creative Commons Attribution 4.0 International License.

HotStorage '26, Prague, Czech Republic

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2902-7/26/09
https://doi.org/10.1145/3837053.3837349

Keywords

LLM, token write amplification, semantic coalescing

ACM Reference Format:
Shijun Yang, Shuheng Gao, Chaomei Yan, Guoli Wei, and Yongkun Li. 2026. Token Write Amplification in LLM-Mediated State Stores. In *18th ACM Workshop on Hot Topics in Storage and File Systems (HotStorage '26), September 28–29, 2026, Prague, Czech Republic.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3837053.3837349

1 Introduction

LLM-based applications [3, 20, 21] are increasingly stateful systems that accumulate context over time. For example, a deep research agent may perform many web searches while preserving useful evidence from earlier steps. These applications create a new storage workload: maintaining useful application state as context grows. Storage systems often apply lightweight preprocessing before persistence, such as parsing or normalization [16]. Vector-based RAG [8] similarly stores embedded chunks for retrieval, but can be insufficient to recover state needed for long term tasks as application state grows [5, 14]. This motivates LLM-mediated state stores [4, 6, 9, 18], which use LLMs on writes to interpret raw context and update durable semantic state. As illustrated in Figure 1, their write paths go beyond indexing: LLMs extract application semantics and consolidate them with existing state.

Putting LLM on the write path makes the cost of a write depend not only on bytes written, but also on LLM tokens spent to maintain state. A small write can trigger much larger LLM work to interpret the raw context and update the maintained state. We call this effect Token Write Amplification (TWA): the number of LLM tokens consumed per token of raw context written on the write path. It is a token-level analog of write amplification in storage systems [1, 10, 11].

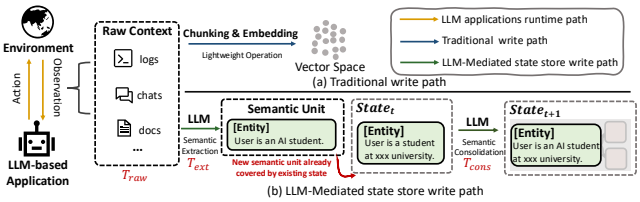
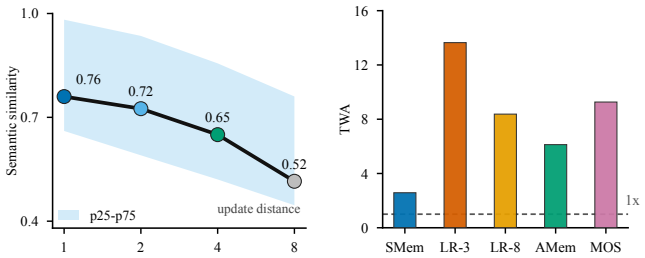


Figure 1: LLM-mediated state stores transform raw context into structured state.



(a) Temporal locality **(b) TWA**

Figure 2: Token write amplification in state stores and temporal locality in application write streams.

However, this amplified token cost does not come completely from maintaining new application semantics. In LLM-mediated state stores, a later write may introduce new semantics, but it may also repeat, revise, or extend prior semantics already represented in the state. Such revisits can force the write path to spend tokens on content that is not wholly new to the maintained state, creating a semantic-overlap component of the write path cost.

To analyze semantic overlap across different state stores, we model the maintained state using semantic units: atomic pieces of application semantics that a store preserves. This abstraction allows us to compare the semantics of an incoming write with those already present in the maintained state, without relying on the internal format of a store. Building on this abstraction, we define the Semantic Overlap Ratio (SOR) and overlap-attributed TWA as diagnostics for estimating how much token cost comes from already covered semantics.

We evaluate five configurations of four LLM-mediated state stores across seven representative agent workloads. Our results reveal substantial token amplification and semantic overlap. To reduce this cost, we introduce a lightweight, backend-agnostic Semantic Coalescing Buffer (SCB) that groups related writes before storage.

We make three main contributions. First, we identify stateful LLM applications as an emerging storage workload and show that LLM-mediated state stores shift write-path cost from bytes written to tokens processed. Second, we introduce Token Write Amplification (TWA), together with semantic-unit accounting, to compare heterogeneous state stores and attribute amplification to semantics already covered by the maintained state. Third, across five store configurations and seven workloads, we measure an average native TWA of 8.00×, associate 54.9% of it with already-covered semantics, and show that a backend-agnostic Semantic Coalescing Buffer reduces average TWA to 3.02×.

2 Motivation

LLM applications often generate write streams with strong semantic locality. As the application interacts with its environment, nearby writes frequently come from the same ongoing task and revisit related information. For example, a deep research agent may retrieve several pages that provide similar evidence about the same topic. Figure 2a shows this effect in our traces: writes that are closer in time have higher semantic similarity. Since state stores accumulate state from earlier writes, such locality suggests that later writes may overlap with semantics already captured by the store.

This overlap matters because LLM-mediated state stores invoke LLMs on the write path. Unlike traditional write paths that mainly tokenize, chunk, and embed raw context, these stores extract semantic units and consolidate them with existing state. Thus, a small raw-context write can trigger much larger LLM input and output to maintain the semantic state. Figure 2b shows that representative state stores often spend several to tens of LLM tokens per raw context token written. We call this effect token write amplification (TWA), a token-level analogue of write amplification in storage systems.

However, aggregate token counts show that writes are expensive, not whether the cost comes from incorporating new semantics or revisiting state the store already maintains. Chunk-level similarity indicates temporal locality, but it does not measure how much of an incoming write is already covered by the maintained state. This motivates metrics that measure write-path token amplification and estimate how much of it is associated with already covered semantics.

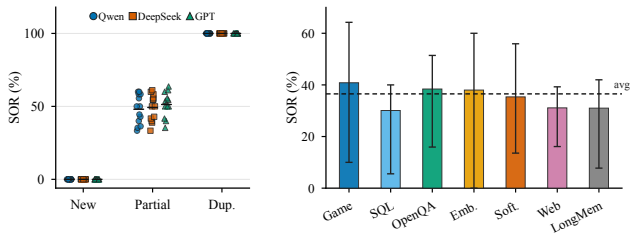
3 Measuring Token Amplification

3.1 System Model

We model an LLM-mediated state store as a storage component with a write interface. For the w-th write, the application submits raw context C_w with $T_{\text{raw}}(w)$ tokens, and the store updates durable semantic state from S_w to S_{w+1}:

$$S_{w+1} = \text{Write}(S_w, C_w).$$

We abstract this store along two dimensions: the semantic content it maintains and the LLM operations triggered by each write. To measure stores with different internal representations, we use *semantic units*: logical state items that



(a) Controlled cases. **(b) Real workloads.**

Figure 3: SOR separates controlled overlap cases and reveals semantic overlap across real workloads.

capture the smallest self-contained pieces of application semantics preserved by the store. This abstraction separates maintained semantics from their underlying representation. Appendix A provides additional details on semantic units.

With this abstraction, the write path of LLM-mediated state store can be described as two LLM-driven stages. *LLM extraction* derives semantic units from the incoming context:

$$U(w) = \text{Extract}(C_w) = \{u_1, u_2, \ldots, u_n\}.$$

LLM consolidation incorporates these units into the existing state S_w. The tokens consumed by these stages are internal write path cost, separate from the raw context submitted by the application.

3.2 Token Level Amplification

With this model in place, we now define *Token Write Amplification*, which measures the write path LLM token consumption per token of raw context written. We also report *Token Space Amplification* (TSA) as a secondary diagnostic for the overhead of the maintained state. Table 5 summarizes the notation used in these definitions.

Token counts include all model input after prompt construction and all generated output. For a write w,

$$\text{TWA}(w) = \frac{T_{\text{ext}}(w) + T_{\text{cons}}(w)}{T_{\text{raw}}(w)}.$$

TWA measures write-path token cost, but reducing write path work can shift cost into the stored state. We therefore use TSA as a secondary diagnostic. At time t,

$$\text{TSA}(t) = \frac{T_{\text{stored}}(t)}{T_{\text{sem}}(t)}.$$

A larger TSA indicates more storage overhead, often due to redundant or fragmented state representations. Appendix E.2 describes the implementation details of TSA measurement.

3.3 Semantic Overlap

TWA quantifies write path token cost, but it does not reveal whether the cost comes from incorporating new semantics or just from revisiting semantics already covered by the maintained state. This distinction matters because already covered semantics can still trigger LLM extraction and consolidation. We measure this effect with the *Semantic Overlap Ratio* (SOR).

For each unit $u \in U(w)$, let $\text{residual}(u, S_w)$ be the semantic content in u that is not yet covered by S_w. We define

$$\text{SOR}(w) = 1 - \frac{\sum_{u \in U(w)} \text{tokens}(\text{residual}(u, S_w))}{\sum_{u \in U(w)} \text{tokens}(u)}.$$

SOR is therefore the fraction of the semantic content of the write already covered by the existing state. In our measurements, we use an LLM to estimate each residual following the definition above. Appendix E.1 gives the prompt.

To relate SOR to token cost, we define *overlap-attributed TWA*, denoted $\text{TWA}^{\text{overlap}}$, as an estimate of the write path token cost associated with semantics already covered by the existing state. For extraction, we attribute an $\text{SOR}(w)$ fraction of the extraction cost to overlap, because extraction processes the full incoming context. For consolidation, we separate tokens into selected existing state tokens, newly extracted semantic unit tokens, and output tokens. We treat all selected existing state tokens as overlap-related because they have already been processed by LLM in earlier writes. We multiply the newly extracted semantic unit token count by $\text{SOR}(w)$ to estimate the overlap portion of these tokens. For output tokens, we use input proportional attribution: the overlap-attributed share of the consolidation input is applied to the consolidation output. Appendix B gives the full accounting rule.

3.4 Controlled Validation of SOR

Because SOR depends on both semantic unit extraction and residual estimation, we validate the full SOR pipeline using controlled cases with known overlap. We generate 45 controlled cases from 15 unrelated base facts, as detailed in Appendix C. In each case, the state contains a fact A, and the incoming chunk is either a restatement of A, a mixture of A and an unrelated fact B, or only B. The chunks include noise and are processed by the same SOR pipeline used in the evaluation.

Figure 3a shows that SOR separates these cases across three judge models: new chunks are close to 0, duplicates are close to 100%, and partial overlap chunks fall between them. This sanity check suggests that the residual-based procedure is robust to the choice of judge model and can distinguish the main overlap cases used in our analysis.

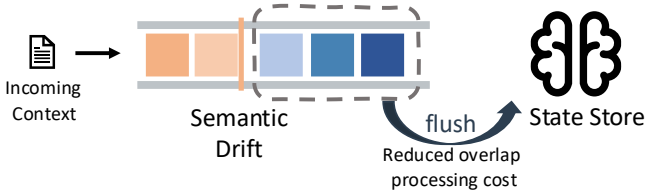


Figure 4: SCB coalesces semantically related writes before flushing to the state store.

3.5 Semantic Coalescing Buffer

The temporal locality in Figure 2a suggests that nearby writes often contain semantically related updates to the same evolving task state. We therefore introduce a lightweight *Semantic Coalescing Buffer* (SCB). As shown in Figure 4, SCB buffers related raw-context writes and flushes them as one concatenated backend write when semantic continuity breaks or a capacity limit is reached.

SCB estimates semantic continuity using embeddings of raw contexts. For an incoming context (C) and an active buffer (B), it computes

$$\mathrm{cont}(C, B) = \max_{C_j \in \mathrm{tail}_k(B)} \cos\big(\mathrm{Embed}(C), \mathrm{Embed}(C_j)\big).$$

SCB maintains a history (H) of recent continuity scores for adaptive boundary detection. The buffer is flushed when the current continuity score falls below this recent history by an adaptive threshold, or when adding C would exceed either the write-count limit or the raw-token limit. Algorithm 1 summarizes the policy.

SCB changes only write grouping. Because buffered contexts remain raw until flush time, SCB delays extraction and consolidation and invokes the backend write path once per group rather than per individual write. This amortizes extraction overhead and avoids intermediate consolidations against the existing state, reducing TWA without changing the underlying state-store interface. For durability, buffered raw contexts can be persistently logged and replayed after crashes.

During buffering, SCB exposes a two-tier read view: the consolidated state store provides persisted state, while the active raw-context buffer provides recent unflushed writes. This increases read-side context and mixes raw and consolidated representations. Section 4 evaluates these effects.

Algorithm 1 Semantic Coalescing Buffer

procedure SCB($\mathcal{W}, \mathcal{M}$)
  $B \leftarrow [\,]$
  $H \leftarrow [\,]$
  for each raw context C in $\mathcal{W}$ **do**
    $boundary \leftarrow$ false
    if B is not empty **then**
      $s \leftarrow \mathrm{cont}(C, B)$
      $boundary \leftarrow \textsc{Boundary}(s, H)$
      append s to H
    end if
    $overflow \leftarrow$ adding C exceeds the write-count or raw-token limit
    if B is not empty and ($boundary$ or $overflow$) **then**
      $\mathcal{M}$.Write(Concat(B))
      $B \leftarrow [\,]$
    end if
    append C to B
  end for
  if B is not empty **then**
    $\mathcal{M}$.Write(Concat(B))
  end if
end procedure

4 Evaluation

4.1 Experimental Setup

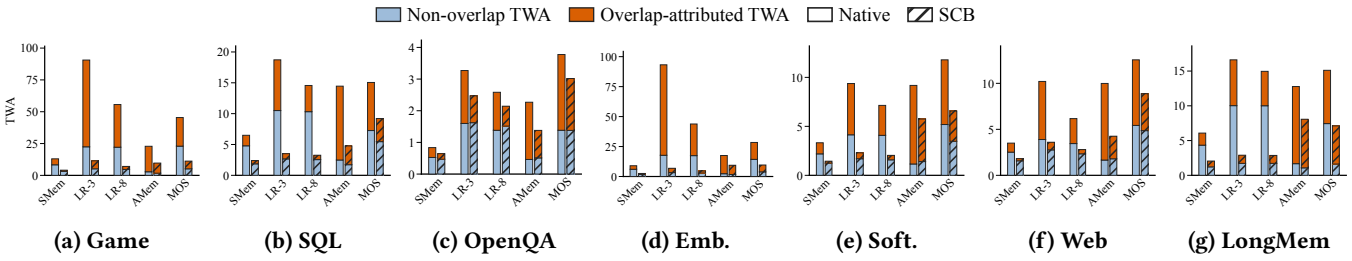
LLM-mediated state stores. We evaluated SimpleMem [9], LightRAG [4], A-Mem [18], and MemoryOS [6]. These backends use different semantic units, ranging from compact memory entries and graph elements to structured notes and hierarchical memory items. Table 4 gives the mapping for each backend. For LightRAG, we report LR-3 and LR-8, with `FORCE_LLM_SUMMARY_ON_MERGE` set to 3 and 8. Larger values invoke LLM summarization during merge less frequently. We use *native* for each store's default write path without SCB.

Workloads. We evaluate on two benchmarks: AMA-Bench and LongMemEval [17, 23]. We replay AMA-Bench task traces as ordered action-observation writes and LongMemEval conversations as writes over user-assistant turns. In total, the evaluation covers 184 streams, 16.6K writes, and 12.2M raw context tokens. Table 7 summarizes the workloads.

Metrics and implementation. We report TWA, TSA, SOR, overlap-attributed TWA, read path tokens per query, and read side accuracy. Accuracy is computed using the official evaluation scripts for the corresponding benchmarks. The main experiments use Qwen3-8B [19] served with vLLM [7] on NVIDIA A40 GPUs, and SCB uses Qwen3-Embedding-0.6B [22] for similarity. For SCB, we set the embedding look-back window to $k = 3$, the similarity-history window to $W = 12$, the drift threshold to $\tau = 2.0$, and the buffer limits to $N_{\max} = 10$ writes and $T_{\max} = 8192$ raw-context tokens, using the same settings across all workloads and backends. Appendix E provides implementation details.

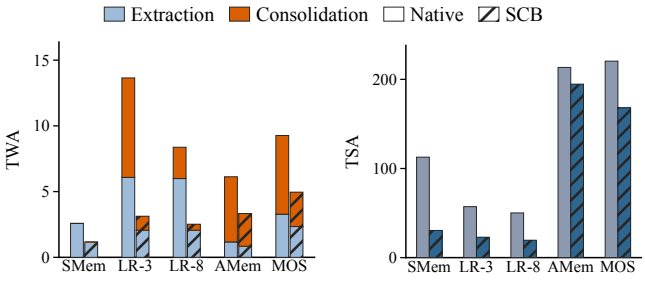
4.2 Results

Semantic overlap is common and costly in our accounting. Figure 3b shows that semantic overlap is common across



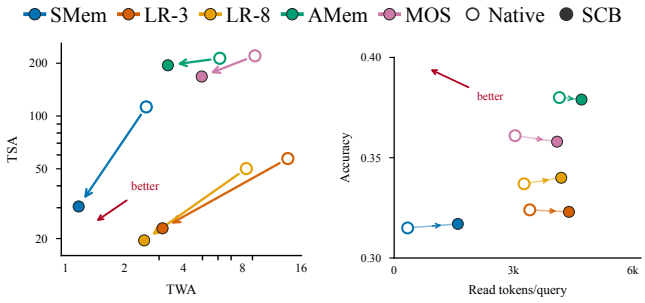
(a) Game (b) SQL (c) OpenQA (d) Emb. (e) Soft. (f) Web (g) LongMem

Figure 5: TWA breakdown by semantic overlap across workloads and state stores, with and without SCB.



(a) TWA breakdown. (b) TSA.

Figure 6: SCB reduces both TWA and TSA.



(a) Write vs. space cost. (b) Read cost vs. accuracy.

Figure 7: Cost tradeoffs across write, space, and read.

workloads: the average SOR is 36.8% across workloads, although individual writes vary substantially. Figure 5 shows that this overlap is costly in our accounting. Across native systems, overlap-attributed accounting associates 54.9% of total TWA with semantics already covered by the state. Aggregate token counts hide this distinction: they show that writes are expensive, but not whether the store is incorporating new semantics or revisiting already maintained content.

Workload structure drives differences in amplification. Figure 5 shows that native TWA differs substantially across workloads, reflecting two write patterns. Short, dense workloads, such as Game in Figure 5a, can trigger extraction and consolidation for many small writes. Long, noisy workloads, such as Openworld-QA in Figure 5c, amortize token cost over larger raw inputs that contain sparse relevant information. This difference yields much higher native TWA in Game than in Openworld-QA (45.51× vs. 2.55×), suggesting that workload structure strongly affects token amplification.

State representation affects amplification. Different stores maintain semantic content in different forms, affecting both token cost on the write path and the number of tokens in the maintained state. Figure 6a decomposes total TWA into extraction and consolidation costs. For most stores, Consolidation accounts for much of the TWA because write-path maintenance repeatedly brings existing state back into the LLM. Figure 6b shows that richer representations such as A-Mem and MemoryOS materialize more tokens per maintained semantic unit, leading to higher TSA. LightRAG further shows that more frequent LLM consolidation does not always reduce TSA, suggesting that LLM-mediated consolidation is not equivalent to traditional compaction.

Native stores expose cost tradeoffs across paths. Figure 7a shows that no native store achieves both low TWA and low TSA. SimpleMem keeps TWA low by performing only extraction, but retains a more redundant state and thus has a higher TSA. LightRAG trades higher write path cost from compaction for a lower TSA, while A-Mem and MemoryOS maintain a richer state and increase both costs. Figure 7b shows the read side tradeoff: SimpleMem retrieves fewer tokens but is less accurate, whereas higher-accuracy stores expose more tokens per query.

Semantic coalescing reduces amplification without changing state stores. SCB reduces TWA by batching raw contexts before they enter state maintenance. This reduces the number of LLM write calls, amortizes fixed prompt overhead across related writes, and avoids repeated consolidations against intermediate states. As a result, SCB reduces average TWA from 8.00× to 3.02×, lowering both overlap-attributed and non-overlap TWA, while also reducing TSA by 33%. Overall, SCB shows that write scheduling alone can reduce write-path token cost by exploiting semantic locality, without changing the underlying state-store interface.

Table 1: End-to-end SCB cost relative to Native.

Backend	Δ write tok.	Δ write lat.	Δ read tok.	Δ read lat.
SMem	−1.1k	−19.3s	+1.0k	+0.2s
LR-3	−8.3k	−31.0s	+1.0k	+0.1s
LR-8	−4.6k	−18.5s	+0.9k	+0.2s
A-Mem	−2.2k	−9.8s	+0.7k	−0.1s
MOS	−3.4k	−11.8s	+1.0k	0.0s

Table 2: SOR sensitivity to judge model.

Workload	Qwen3-8B	Qwen3-14B	Qwen3-32B	DS-V4-Pro
LongMem.	30.87%	31.93%	33.98%	28.07%
Soft.	34.76%	31.27%	31.51%	30.94%
OpenQA	37.47%	37.94%	40.94%	39.20%

Table 3: TWA sensitivity across write-path models, reported as mean ± maximum deviation over Qwen3-8B, Qwen3-14B, Qwen3-32B, and DeepSeek-V4-Pro.

Backend	Workload	Native	Overlap	SCB
LR-3	LongMem.	16.6 ± 0.5	6.2 ± 0.4	2.9 ± 0.3
LR-3	Soft.	9.3 ± 0.7	5.2 ± 0.3	2.3 ± 0.2
A-Mem	LongMem.	12.7 ± 0.4	11.1 ± 0.1	8.1 ± 0.1
A-Mem	Soft.	9.2 ± 0.3	8.0 ± 0.1	5.8 ± 0.1

Read cost of exposing buffered writes. SCB increases read tokens because unflushed raw contexts must remain visible before they enter state maintenance. Figure 7b shows that this extra context does not noticeably affect downstream accuracy, since SCB preserves the persisted state and only appends the active raw-context buffer. SCB bounds the size of this added context by flushing on semantic boundaries, write-count limits, or raw-token limits. Table 1 shows that SCB adds 0.7k–1.0k read tokens per query, compared with 1.1k–8.3k fewer write tokens per update. Thus, write-side token savings exceed the extra read-side tokens needed to expose buffered writes. Because the read-token increase is small, end-to-end read latency changes little, by at most 2.2%.

Model sensitivity. Tables 2 and 3 report model sensitivity for SOR and TWA. For SOR, we fix the residual estimation procedure and vary only the judge model. The estimates remain close across models, suggesting that semantic overlap is mainly a property of the write streams rather than an artifact of any particular judge model. For TWA, we fix the workload stream and backend implementation and vary only the write-path LLM. Native TWA and TWA attributed to overlap remain close, showing that token amplification is primarily driven by workload redundancy and how each backend maintains state.

5 Discussion

Token read amplification. A natural read side analog is *Token Read Amplification* (TRA): the number of state tokens exposed to a query relative to the minimum semantic evidence needed to answer it. However, unlike TWA, TRA heavily depends on retriever design, making it harder to define in a way that applies across different stores. We leave a full treatment of TRA for future work.

Lightweight overlap estimation. SOR is an offline diagnostic metric rather than an online signal, because computing it still requires semantic unit extraction and residual estimation. A promising direction is to use offline SOR labels to train cheaper predictors that estimate whether incoming raw context overlaps with existing state, enabling overlap-aware policies that delay, coalesce, or filter writes before expensive extraction and consolidation.

Intra-write redundancy. Semantic coalescing groups nearby writes but does not remove redundancy within a single write. This limits its benefit on workloads such as Openworld-QA, where a write may contain long, noisy, repetitive web search context. Future designs could add processing within each write, such as local filtering, to expose overlap that similarity across writes cannot capture.

LLM calls as internal storage operations. LLM-mediated state stores are one instance of a broader shift in storage systems: LLM calls are becoming internal operations rather than external application logic. Database systems illustrate this trend, with recent designs exploring LLMs as operators for semantic data processing [13, 15]. Although these systems target different workloads, they share the same cost shift: LLM token consumption becomes an internal storage-system cost. This suggests that token amplification may be useful wherever LLM calls enter storage-system internals.

6 Conclusion

LLM-mediated state stores make LLM operations part of the storage write path, turning token consumption into a storage system cost. We introduced TWA to quantify write path token cost, with semantic unit accounting enabling cross store comparison and overlap attribution. Our measurements show that representative stores spend many tokens on semantics already covered by existing state, and that simple semantic coalescing can substantially reduce this amplification without backend changes. These results suggest that future LLM-mediated state stores should explicitly consider token amplification and semantic overlap when designing their write paths.

Acknowledgments

This work was supported by NSFC (62472392). Yongkun Li is the corresponding author.

References

[1] Niv Dayan and Stratos Idreos. 2018. Dostoevsky: Better Space-Time Trade-Offs for LSM-Tree Based Key-Value Stores via Adaptive Removal of Superfluous Merging. In *Proceedings of the 2018 International Conference on Management of Data*. 505–520.

[2] DeepSeek-AI. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence.

[3] Yihong Dong, Xue Jiang, Jiaru Qian, Tian Wang, Kechi Zhang, Zhi Jin, and Ge Li. 2025. A survey on code generation with llm-based agents. *arXiv preprint arXiv:2508.00083* (2025).

[4] Zirui Guo, Lianghao Xia, Yanhua Yu, Tian Ao, and Chao Huang. 2024. Lightrag: Simple and fast retrieval-augmented generation. *arXiv preprint arXiv:2410.05779* 2, 3 (2024).

[5] Yuyang Hu, Shichun Liu, Yanwei Yue, Guibin Zhang, Boyang Liu, Fangyi Zhu, Jiahang Lin, Honglin Guo, Shihan Dou, Zhiheng Xi, et al. 2025. Memory in the age of ai agents. *arXiv preprint arXiv:2512.13564* (2025).

[6] Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. 2025. Memory os of ai agent. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*. 25972–25981.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*. 611–626.

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. 2020. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* 33 (2020), 9459–9474.

[9] Jiaqi Liu, Yaofeng Su, Peng Xia, Siwei Han, Zeyu Zheng, Cihang Xie, Mingyu Ding, and Huaxiu Yao. 2026. SimpleMem: Efficient Lifelong Memory for LLM Agents. *arXiv preprint arXiv:2601.02553* (2026).

[10] Lanyue Lu, Thanumalayan Sankaranarayana Pillai, Andrea C. Arpaci-Dusseau, and Remzi H. Arpaci-Dusseau. 2016. WiscKey: Separating Keys from Values in SSD-conscious Storage. In *14th USENIX Conference on File and Storage Technologies (FAST 16)*. 133–148.

[11] Patrick O'Neil, Edward Cheng, Dieter Gawlick, and Elizabeth O'Neil. 1996. The log-structured merge-tree (LSM-tree). *Acta Inf.* 33, 4 (June 1996), 351–385. doi:10.1007/s002360050048

[12] OpenAI. 2025. Introducing GPT-5. https://openai.com/index/introducing-gpt-5/.

[13] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2024. Semantic operators: a declarative model for rich, ai-based data processing. *arXiv preprint arXiv:2407.11418* (2024).

[14] Boci Peng, Yun Zhu, Yongchao Liu, Xiaohe Bo, Haizhou Shi, Chuntao Hong, Yan Zhang, and Siliang Tang. 2025. Graph retrieval-augmented generation: A survey. *ACM Transactions on Information Systems* 44, 2 (2025), 1–52.

[15] Kangkang Qi, Dongyang Xie, Wenbo Li, Hao Zhang, Yuanyuan Zhu, Jeffrey Xu Yu, and Kangfei Zhao. 2026. Sema: A High-performance System for LLM-based Semantic Query Processing. *arXiv preprint arXiv:2603.11622* (2026).

[16] Erhard Rahm et al. 2000. Data cleaning: Problems and current approaches. (2000).

[17] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. 2024. Longmemeval: Benchmarking chat assistants on long-term interactive memory. *arXiv preprint arXiv:2410.10813* (2024).

[18] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. 2026. A-mem: Agentic memory for llm agents. *Advances in Neural Information Processing Systems* 38 (2026), 17577–17604.

[19] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).

[20] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2022. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629* (2022).

[21] Wenlin Zhang, Xiaopeng Li, Yingyi Zhang, Pengyue Jia, Yichao Wang, Huifeng Guo, Yong Liu, and Xiangyu Zhao. 2025. Deep research: A survey of autonomous research agents. *arXiv preprint arXiv:2508.12752* (2025).

[22] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. 2025. Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models. *arXiv preprint arXiv:2506.05176* (2025).

[23] Yujie Zhao, Boqin Yuan, Junbo Huang, Haocheng Yuan, Zhongming Yu, Haozhou Xu, Lanxiang Hu, Abhilash Shankarampeta, Zimeng Huang, Wentao Ni, et al. 2026. Ama-bench: Evaluating long-horizon memory for agentic applications. *arXiv preprint arXiv:2602.22769* (2026).

A Notation and Semantic Units

We abstract maintained state as semantic units: logical units of application semantics, such as entities, relations, facts, claims or notes. Concrete state stores may materialize and organize these units in different forms, including sets of graphs, summaries, or hierarchical memories. Table 4 maps the evaluated backends to this abstraction. Despite their different concrete state representations, all backends expose the same measurement interface: raw-context writes, semantic extraction, and either appending or consolidation into maintained state.

Table 5 summarizes the notation used in the system model and metrics.

B Overlap-attributed TWA Details

We use overlap-attributed TWA as an estimate of how many write path tokens are associated with semantics already covered by existing state. The goal is not to identify the exact contribution of each token, but to apply a consistent rule for comparing overlap-induced work across stores and workloads. We compute this attribution separately for LLM extraction and LLM consolidation.

During LLM extraction, the store processes the full raw context even when only part of the context is semantically new. We therefore attribute extraction cost in proportion to the Semantic Overlap Ratio of the write:

$$\mathrm{TWA}_{\mathrm{ext}}^{\mathrm{overlap}}(w) = \frac{\mathrm{SOR}(w) \cdot T_{\mathrm{ext}}(w)}{T_{\mathrm{raw}}(w)}.$$

For LLM consolidation, the token cost comes from three components: selected existing state tokens $T_{\mathrm{cons}}^{\mathrm{sel}}(w)$, newly extracted semantic-unit tokens $T_{\mathrm{cons}}^{\mathrm{new}}(w)$, and output tokens $T_{\mathrm{cons}}^{\mathrm{out}}(w)$. We attribute selected existing state tokens to overlap because they correspond to state that has already been processed by LLM before the current write. For newly extracted semantic units, we attribute cost in proportion to $\mathrm{SOR}(w)$. This gives the overlap-attributed consolidation input:

$$A_{\mathrm{in}}(w) = T_{\mathrm{cons}}^{\mathrm{sel}}(w) + \mathrm{SOR}(w) \cdot T_{\mathrm{cons}}^{\mathrm{new}}(w).$$

We then attribute consolidation output tokens in proportion to the overlap-attributed share of the consolidation input:

$$T_{\mathrm{cons}}^{\mathrm{overlap}}(w) = A_{\mathrm{in}}(w) + \frac{A_{\mathrm{in}}(w)}{T_{\mathrm{cons}}^{\mathrm{sel}}(w) + T_{\mathrm{cons}}^{\mathrm{new}}(w)} \cdot T_{\mathrm{cons}}^{\mathrm{out}}(w).$$

The resulting overlap-attributed TWA for write w is

$$\mathrm{TWA}^{\mathrm{overlap}}(w) = \mathrm{TWA}_{\mathrm{ext}}^{\mathrm{overlap}}(w) + \frac{T_{\mathrm{cons}}^{\mathrm{overlap}}(w)}{T_{\mathrm{raw}}(w)}.$$

C Controlled SOR Validation Details

The controlled SOR validation used in Figure 3(a) evaluates whether the SOR pipeline can distinguish controlled cases with different levels of semantic overlap: new, partially overlapping, and duplicate semantics.

Seed facts. We use 15 mutually unrelated seed facts covering the main workload domains used in our evaluation. Each fact is paired with a short paraphrase used to construct duplicate and partial overlap chunks.

Case generation. For each seed fact A_i, we pair it with the next seed fact $B_i = A_{i+1}$, using wraparound for A_{15}. This gives 15 (A, B) pairs. For each pair, we construct three cases:

- **Duplicate**: the state contains A, and the incoming chunk restates the same fact using its short paraphrase.
- **Partial overlap**: the state contains A, and the incoming chunk contains the short paraphrase of A and the unrelated new fact B.
- **New**: the state contains A, and the incoming chunk contains only the unrelated fact B.

Each incoming chunk is wrapped with noise before being processed by the same SOR pipeline used in the main experiments. The fixed noise templates include tool-log headers and numeric strings. SOR is computed over the extracted semantic units and their residuals. The noise is included only to test whether the pipeline ignores irrelevant context.

Example. For A = `f01` and B = `f02`, the controlled state is "user prefers concise answers." The three clean incoming chunks are:

- Duplicate: User wants brief answers.
- Partial: User wants brief answers. Project uses Python 3.11.
- New: Project uses Python 3.11.

Judge models. We repeat residual estimation with three judge models: Qwen3-8B, DeepSeek-V4-Pro [2], and GPT-5-mini [12]. We keep the SOR pipeline fixed and vary only the judge model.

D Workloads

Table 7 summarizes the workloads used in the evaluation. AMA-Bench tasks are replayed as ordered action-observation writes, while LongMemEval is replayed as conversation-turn writes. We select streams with more than 20 interactions to ensure sufficient state accumulation in each evaluated stream.

Table 4: Mapping evaluated baselines to the unified LLM-mediated state store abstraction.

Backend	Semantic units	Extraction stage	Consolidation stage
SimpleMem	Compact memory entries.	LLM derives semantic memory entries from the raw context.	Entries are appended to memory without consolidation.
LightRAG	Entities and relations.	LLM extracts graph elements from the raw context.	Description of each element may be summarized by the LLM.
A-Mem	Memory notes with structured attributes.	LLM constructs candidate notes from the raw context.	Notes are linked to related memories and may update existing notes.
MemoryOS	Dialogue pages, topic segments, and persona/KB/trait entries.	LLMs generate STM page metadata by determining dialogue linkage.	LLMs summarize MTM segments and update LPM knowledge entries.

Table 5: Notation used in the system model.

Symbol	Meaning
S_w	State before the w-th write.
C_w	Raw context received by the w-th write.
$U(w)$	Semantic units introduced by the w-th write.
$T_{\text{raw}}(w)$	Number of raw context tokens in C_w.
$T_{\text{ext}}(w)$	Token cost for extracting semantic units from C_w.
$T_{\text{cons}}(w)$	Token cost for consolidating extracted semantic units into existing state.
$T_{\text{stored}}(t)$	Number of semantic-state tokens actually stored at time t.
$T_{\text{sem}}(t)$	Minimum tokens needed to express the maintained semantic content at time t.

Table 6: Seed facts used for controlled chunk-level SOR validation.

ID	Domain	Base fact	Short paraphrase
f01	Conversation	user prefers concise answers	User wants brief answers.
f02	Coding	project uses Python 3.11	Project runs on Python 3.11.
f03	Coding	parser.py handles empty input strings	parser.py supports empty strings.
f04	Web search	searched laptop model supports USB-C charging	The laptop supports USB-C charging.
f05	Game	player is in kitchen	Player is in the kitchen.
f06	Embodied AI	robot faces red block	Robot faces the red block.
f07	Text-to-SQL	orders table contains order_id column	orders has an order_id column.
f08	Data analysis	dataset has missing values in age column	age column has missing values.
f09	Tool use	calculator tool returned result 42	Calculator returned 42.
f10	Planning	meeting scheduled for Friday at 3 PM	Meeting is Friday at 3 PM.
f11	Preference	user rejects seafood restaurant recommendations	User does not want seafood restaurants.
f12	File system	report saved at /home/user/report.pdf	Report is at /home/user/report.pdf.
f13	API	API endpoint is /v1/search	Search endpoint is /v1/search.
f14	Debugging	test_login fails with timeout error	test_login times out.
f15	Navigation	target store located on second floor	Target store is on floor two.

E Implementation details

E.1 Residual Estimation Prompt

This prompt estimates residual semantic content for SOR: given the current state and one semantic unit, the model emits only the part of that unit that is not already recoverable from the state. We then compute SOR from the token length of the original unit and the residual.

Residual Estimation Prompt:

> You are estimating the residual semantic content of a semantic unit with respect to the current state.

Table 7: Workload summary. Raw tokens count the full replay.

Workload	Type	Trace CNT	Avg. updates/trace	Avg. tokens/update
Game	Text-game action and observation traces	29	154.7	103
Text2SQL	Database-question and SQL-generation traces	29	23.02	294
Openworld-QA	Web-search and open-domain QA traces	27	44.2	5.91K
Embodied AI	Robot/planning action–observation traces	30	120.3	171
Software	Coding, debugging, and tool-use traces	36	103.2	639
Web	Browser-navigation and web-task traces	23	28.9	1.17K
LongMemEval	Long conversational memory traces	10	222.0	303
Total	–	184	90.3	733

Current state before the update: {current_state if current_state.strip() else "(empty)"}
Current semantic unit: {unit_text}
Task: 1. Output the minimal residual text: the information in the current semantic unit that is not recoverable from the current state. 2. If the current state fully covers the semantic unit, output an empty residual string. 3. If the current state provides no useful coverage, output a compressed residual that preserves the full useful meaning of the semantic unit. 4. Keep exact names, files, commands, numbers, dates, versions, and outcomes. Do not remove information needed to understand the residual independently. 5. The current state plus the residual should preserve the useful meaning of the current semantic unit. 6. Do not explain your decision. Use plain English only. 7. Return compact valid JSON only, with no markdown:
{"residual":""}

E.2 Details of TSA

This section describes the implementation details for computing TSA. TSA needs $T_{\text{sem}}(t)$, the minimum token count needed to express the semantic content maintained up to time t. We estimate this with an append-style reference state that is independent of any backend implementation:

(1) Start from an empty reference state R_0.
(2) For each write w, append the newly extracted semantic units $U(w)$ to the previous reference state R_w.
(3) Ask the LLM to merge, deduplicate, and update the reference state, yielding R_{w+1}.
(4) Tokenize R_{w+1} to obtain $T_{\text{sem}}(w+1)$, the TSA denominator at that point.

This reference state is used only for measurement; it is not passed to or stored by the evaluated state stores.
Reference State Construction Prompt:

You are producing a compact reference representation used to estimate the minimum tokens needed to express the maintained semantic content.
Previous reference state: {previous_state if previous_state.strip() else "(empty)"}
New semantic units: {unit_text}
Task: 1. Merge the new semantic units into the previous reference state. 2. Keep only distinct semantic content that remains valid after the update. 3. If a new unit supersedes an existing fact, keep the latest valid version; keep older values only when they remain meaningful as history. 4. Deduplicate facts that express the same meaning, even if wording differs. 5. Express each retained fact in the shortest clear form. Keep exact names, paths, commands, numbers, dates, versions, and outcomes when they are part of the meaning. 6. Remove raw boilerplate, repeated observations, and formatting artifacts. 7. Do not explain your decision. Use plain English only. 8. Return compact valid JSON only, with no markdown: {"state":"one compact line per retained semantic fact"}